

تقرير مشروع

التعرف الضوئي على حروف مخطوطات اللغة الروسية القديمة باستخدام YOLO

نظام OCR للتعرف على الحروف واستخراج النص من صور اللغة الروسية
القديمة

إعداد :

رقية حسن احمد النزيلي

إشراف :

د. اكرم الصباري

فهرس المحتويات

- ١- مقدمة
- ٢- فكرة المشروع
- ٣- إعداد الأبجدية
- ٤- إنشاء قاعدة البيانات الاصطناعية
- ٤-١ إنشاء الصور
- ٥- إنشاء صناديق الإحاطة تلقائيًا (Bounding Boxes)
- ٦- محاكاة خصائص المخطوطات القديمة
- ٧- تقسيم البيانات الاصطناعية
- ٨- تحسين سرعة التدريب باستخدام الذاكرة المؤقتة (RAM)
- ٩- التدريب الأولي للنموذج (Pre-training)
- ١٠- استخدام الصور الحقيقية للمخطوطات
- ١١- تجهيز البيانات الحقيقية
- ١٢- تحسين صور المخطوطات الحقيقية
- ١٣- الضبط الدقيق (Fine-tuning) باستخدام البيانات الحقيقية
- ١٤- نموذج النظام النهائي
- ١٥- تطوير واجهة المستخدم باستخدام Gradio
- ١٦- رفع الصور وتحليلها
- ١٧- معالجة وترتيب الحروف
- ١٨- عرض نتيجة الكشف
- ١٩- الكاميرا المباشرة
- ٢٠- إعدادات المستخدم
- ٢١- نشر النظام
- ٢٢- تحويل النظام إلى تطبيق للهاتف
- ٢٣- البنية النهائية للنظام
- ٢٤- النتائج
- ٢٥- التحديات التي عالجها المشروع
- ٢٦- الخلاصة

1. مقدمة :

يهدف هذا المشروع إلى تطوير نظام للتعرف الضوئي على الحروف (Optical Character Recognition – OCR) مخصص للمخطوطات الروسية القديمة، بحيث يستطيع النظام اكتشاف الحروف الموجودة في صور المخطوطات، تحديد مواقعها، ثم إعادة ترتيبها وتحويلها إلى نص قابل للقراءة.

تم الاعتماد في المشروع على تقنيات التعلم العميق، وبشكل أساسي نموذج YOLO لاكتشاف الحروف داخل الصور. ونظرًا لطبيعة المخطوطات القديمة واختلاف أشكال الحروف والخطوط وجودة الصور، تم إنشاء بيانات تدريب اصطناعية بالإضافة إلى استخدام صور حقيقية من المخطوطات في مرحلة لاحقة من التدريب.

مر المشروع بعدة مراحل رئيسية، تبدأ بإعداد الأبجدية والبيانات، ثم إنشاء قاعدة بيانات اصطناعية، وتدريب النموذج عليها، ثم تحسين النموذج باستخدام صور حقيقية للمخطوطات، وأخيرًا إنشاء واجهة استخدام ونشر النظام ليكون متاحًا من خلال الهاتف.

2. فكرة المشروع:

تعتمد فكرة النظام على التعامل مع كل حرف باعتباره فئة مستقلة يمكن لنموذج YOLO اكتشافها داخل الصورة.

عند إدخال صورة لمخطوط، يقوم النظام بالخطوات التالية:

صورة المخطوط → اكتشاف الحروف → تحديد مواقع الحروف → تحديد الفئة الخاصة بكل حرف → ترتيب الحروف → إعادة بناء الأسطر → استخراج النص

ولا يقتصر النظام على اكتشاف الحروف فقط، بل يحتوي أيضًا على مرحلة معالجة لاحقة (Post-processing) يتم فيها ترتيب الحروف وفق مواقعها داخل الصورة وتجميعها في أسطر، ثم إضافة المسافات عند وجود فراغات مناسبة بين الحروف.

3. إعداد الأبجدية:

كانت الخطوة الأولى هي إعداد الأبجدية المستخدمة في المشروع.

تم إنشاء مسار خاص بالمشروع يحتوي على ملف الأبجدية Alphabet.json وملف النص المرجعي corpus.txt بالإضافة إلى مجلد الخطوط.

تم استخدام ملف corpus.txt لاستخراج الحروف الموجودة فعليًا، ثم مقارنتها بالحروف الموجودة في Alphabet.json. تقوم الدالة load_and_sync_alphabet() بتحميل الأبجدية الموجودة، واستخراج الحروف من Corpus، ثم دمج الحروف الجديدة دون تكرار، وإنشاء قاموس يربط كل حرف برقم خاص به char_to_id :

وبذلك أصبح لكل حرف Class ID يستخدم أثناء تدريب نموذج YOLO.

بلغ عدد الفئات النهائية المستخدمة في المشروع 38 فئة.

4. إنشاء قاعدة البيانات الاصطناعية:

نظرًا لصعوبة الحصول على عدد كبير من الصور الحقيقية للمخطوطات مع تحديد مواقع جميع الحروف يدويًا، تم إنشاء Dataset اصطناعية للتدريب الأولي للنموذج.

تم إنشاء حوالي 19,000 صورة اصطناعية.

كان الهدف من إنشاء هذه البيانات هو توفير عدد كبير من الأمثلة المتنوعة لجميع الفئات، مع محاولة الحفاظ على توازن الفئات حتى لا تسيطر بعض الحروف على عملية التدريب.

4.1 إنشاء الصور:

لم تكن الصور الاصطناعية عبارة عن حرف واحد فقط في كل صورة، وإنما تم إنشاء صور تحتوي على عدة حروف، حيث يتراوح عدد الحروف في الصورة الواحدة تقريباً بين 2 و 8 حروف.

تم استخدام مجموعة من الخطوط الموجودة في مجلد fonts ، مع تغيير حجم الخطوط بشكل عشوائي.

كما تم تطبيق اختلافات على شكل الحروف، منها:

- تغيير حجم الحرف.
- تدوير الحرف بدرجات مختلفة.
- تغيير سماكة الخط.
- استخدام أنواع مختلفة من الحبر.
- استخدام ألوان مختلفة للحبر.
- اختيار الخط بشكل عشوائي.
- وضع عدة حروف في الصورة الواحدة.

هذا التنوع يهدف إلى جعل البيانات الاصطناعية أقرب إلى الاختلافات الموجودة في الصور الحقيقية للمخطوطات.

5. إنشاء Bounding Boxes تلقائياً:

من أهم أجزاء إنشاء البيانات الاصطناعية أن مواقع الحروف لم يتم تحديدها يدوياً.

بعد إنشاء كل حرف، تم الاعتماد على البكسلات الفعلية غير الشفافة لتحديد المساحة التي يشغلها الحرف.

ومن خلال هذه المساحة تم إنشاء Bounding Box لكل حرف.

بعد ذلك تم تحويل إحداثيات المربعات إلى الصيغة المطلوبة من YOLO ، وهي الإحداثيات الطبيعية

(Normalized Coordinates) .

وبذلك أصبحت كل صورة اصطناعية مصحوبة بملف Label يحتوي على:

- Class ID للحرف.
- مركز المربع.
- عرض المربع.
- ارتفاع المربع.

وهذا أدى إلى تجهيز البيانات بشكل مباشر للاستخدام في تدريب YOLO .

6. محاكاة خصائص المخطوطات القديمة:

لزيادة واقعية البيانات الاصطناعية، لم يتم وضع الحروف على خلفية بيضاء فقط.

تم إنشاء عدة أنواع من الخلفيات، منها:

- خلفيات تشبه ورق المخطوطات.
- ورق قديم ومصفّر.
- خلفيات ذات Texture.
- تدرجات لونية.
- أسطح تحتوي على ضوضاء.
- خلفيات Vintage.
- خلفيات تشبه الجدران أو الأسطح القديمة.

- كما تم إضافة تأثيرات مختلفة إلى الصور، مثل:
- الخدوش.
- بقع الحبر.
- Gaussian Noise
- Blur
- تغيير السطوع.
- تغيير التباين.

الهدف من هذه العمليات هو محاكاة الظروف غير المثالية التي قد تظهر في صور المخطوطات الحقيقية.

7. تقسيم البيانات الاصطناعية:

تم تقسيم الـ Dataset الاصطناعية البالغ حجمها 19,000 صورة إلى ثلاث مجموعات

المجموعة	النسبة	عدد الصور
Training	70%	13,300
Validation	20%	3,800
Test	10%	1,900
المجموع	100%	19,000

وتم إنشاء ملف data.yaml لتحديد مسارات البيانات وعدد الفئات المستخدمة في تدريب YOLO.

8. تحسين سرعة التدريب باستخدام RAM :

نظرًا لكبر حجم Dataset الاصطناعية وكثرة عمليات قراءة الصور أثناء التدريب، تم نقل نسخة من البيانات إلى مساحة الذاكرة المؤقتة. /dev/shm

تم استخدام هذه المساحة باعتبارها RAM Disk ، مما يساعد على تقليل زمن الوصول إلى ملفات الصور والـ Labels أثناء عملية التدريب.

وبالتالي أصبح مسار التدريب يعتمد على نسخة البيانات الموجودة في الذاكرة بدلًا من الاعتماد الكامل على التخزين الموجود على Google Drive.

9. التدريب الأولي للنموذج Pre-training :

بعد تجهيز Dataset الاصطناعية، بدأت مرحلة التدريب الأولي للنموذج.

تم استخدام نموذج YOLOv8n ، مع تحميل الأوزان الأولية من yolov8n.pt .

وفي حالة وجود تدريب سابق محفوظ، كان الكود قادرًا على استكمال التدريب من last.pt .

أهم إعدادات التدريب :

• عدد الدورات الأقصى (Max Epochs): ١٠٠

• حجم الدفعة (Batch Size): ٣٢

• أبعاد الصورة (Image Size): ٣٨٤ × ٣٨٤

• معدل التعلم (Learning Rate): ٠.٠١

• عدد العمال (Workers): ٨

- استخدام معالج الرسومات (GPU): مفعل
- استخدام الدقة المختلطة (AMP): مفعل
- تفعيل التخزين المؤقت (Cache): مفعل
- تثبيت البذرة العشوائية (Seed): ٤٢

كان الهدف من هذه المرحلة هو تعليم النموذج التعرف على أشكال الحروف المختلفة من خلال البيانات الاصطناعية قبل الانتقال إلى الصور الحقيقية.

10. استخدام الصور الحقيقية للمخطوطات:

بعد الانتهاء من التدريب الأولي، تم الانتقال إلى استخدام بيانات حقيقية مأخوذة من المخطوطات والقطع الأصلية.

بلغ عدد الصور الحقيقية حوالي 223 صورة.

تم تحديد مواقع الحروف داخل الصور الحقيقية يدويًا باستخدام أداة مخصصة للتوسيم، والتي تم الإشارة إليها باسم Labelbox Labeler.

وبذلك أصبحت الصور الحقيقية تحتوي على Bounding Boxes فعلية للحروف، مما يسمح باستخدامها في مرحلة

. Fine-tuning

11. تجهيز البيانات الحقيقية:

تم التحقق أولاً من وجود صورة وملف Label مطابق لكل صورة، والتأكد من أن ملفات الـ Labels تحتوي على بيانات فعلية.

بعد ذلك تم إجراء Shuffle للبيانات باستخدام Seed ثابت قيمته 42، ثم تقسيم البيانات المستخدمة في هذه المرحلة إلى:

• Training : 85%

• Validation : 15%

كما تم تطبيق تحسينات على صور المخطوطات قبل استخدامها في Fine-tuning .

12. تحسين صور المخطوطات الحقيقية:

بسبب طبيعة صور المخطوطات القديمة واحتمالية وجود ضوضاء أو تفاوت في الإضاءة والتباين، تم تطبيق سلسلة من عمليات معالجة الصور.

شملت المعالجة:

- تحويل الصورة إلى Grayscale.
- استخدام fastNIMeansDenoising لتقليل الضوضاء.
- استخدام CLAHE لتحسين التباين المحلي.
- إعادة الصورة إلى صيغة BGR لاستخدامها مع النموذج.

الهدف من هذه المرحلة هو تحسين وضوح الحروف والمساعدة على جعل خصائص الصور الحقيقية أكثر ملاءمة للنموذج.

13. باستخدام البيانات الحقيقية:

بعد تدريب النموذج على البيانات الاصطناعية، تم تحميل أفضل أوزان للنموذج واستخدامها كنقطة بداية لمرحلة

. Fine-tuning

تم استخدام الصور الحقيقية المحسنة لتكييف النموذج مع الخصائص الفعلية للمخطوطات.

إعدادات Fine-tuning تضمنت :

- عدد الدورات (Epochs): 120.
 - أبعاد الصورة (Image Size): 1040.
 - حجم الدفعة (Batch Size): 4.
 - معدل التعلم (Learning Rate): 0.0001.
 - تجميد الطبقات (Freeze): 10 طبقات.
 - معامل الموزاييك (Mosaic): 0.1.
 - وزن خسارة المربع (Box Loss): 7.5.
 - الصبر للتوقف المبكر (Patience): 50.
- وكان الهدف من هذه المرحلة نقل المعرفة التي اكتسبها النموذج من البيانات الاصطناعية إلى الصور الحقيقية للمخطوطات.

14. نموذج النظام النهائي:

بعد إكمال مرحلة Fine-tuning ، تم الاعتماد على ملف الأوزان النهائي best.pt. ويتم استخدام هذا الملف في مرحلة الاستدلال Inference. كما تم استخدام ملف Alphabet.json لتحويل Class IDs التي ينتجها النموذج إلى الحروف المقابلة لها. فعلى سبيل المثال، بدل أن يعرض النظام رقم الفئة فقط، يتم تحويل الرقم إلى الحرف المرتبط به في الأبجدية.

15. تطوير واجهة المستخدم باستخدام Gradio :

تم إنشاء ملف app.py ليكون مسؤولاً عن تشغيل واجهة نظام OCR. تم استخدام مكتبة Gradio لبناء الواجهة، مع مكتبة Ultralytics YOLO لتشغيل النموذج. تحتوي الواجهة على عنوان خاص بنظام التعرف البصري على المخطوطات الروسية القديمة، وتوفر للمستخدمين طريقتين لإدخال الصورة.

16. رفع الصور وتحليلها:

في الخيار الأول يستطيع المستخدم رفع صورة من جهازه. بعد رفع الصورة والضغط على زر « استخراج النص وتحديد الحروف، يتم إرسال الصورة إلى دالة process_image() تقوم الدالة بتحميل النموذج عند الحاجة، ثم تشغيل عملية Prediction باستخدام best.pt وبحجم صورة استدلال . imgsiz = 1056 .

ويتم استخدام قيمة Confidence Threshold يحددها المستخدم.

بعد الانتهاء من الكشف، يتم الحصول على:

• صناديق الإحاطة (Bounding Boxes)

• معرفات الفئات (Class IDs)

• درجات الثقة (Confidence Scores)

ثم يتم رسم المربعات حول الحروف المكتشفة وإظهار الحرف مع نسبة الثقة.

17. معالجة وترتيب الحروف:

بعد أن يقوم YOLO باكتشاف الحروف، تأتي مرحلة مهمة لإعادة بناء النص.

الدالة `group_boxes_into_text()` تستخدم مواقع الـ Bounding Boxes لتحديد الأسطر.

يتم حساب مركز كل Bounding Box على المحور الرأسي، ثم مقارنة مراكز الحروف لتحديد الحروف التي تنتمي إلى السطر نفسه.

بعد تحديد الأسطر، يتم ترتيب الحروف داخل كل سطر وفق موقعها الأفقي.

كما يتم حساب المسافة بين الحروف، وإذا تجاوزت المسافة حدًا معينًا بالنسبة إلى متوسط عرض الحروف، يتم إدخال مسافة في النص الناتج.

وفي النهاية يتم تحويل Class ID إلى الحرف المقابل له باستخدام `Alphabet.json`.

وبذلك يتحول ناتج الكشف من مجموعة من المربعات إلى نص مقروء.

18. عرض نتيجة الكشف:

إلى جانب النص المستخرج، يعرض النظام الصورة الأصلية بعد إضافة Bounding Boxes حول الحروف المكتشفة.

يظهر بجانب كل حرف:

- الحرف المتوقع.
- قيمة Confidence.

وهذا يسمح للمستخدم بمشاهدة نتيجة النموذج بصريًا والتحقق من الحروف التي قام النظام باكتشافها.

19. الكاميرا المباشرة:

بالإضافة إلى رفع الصور، تم تطوير خيار آخر يعتمد على الكاميرا المباشرة.

يستخدم النظام webcam مع خاصية `streaming=True`، ويتم إرسال الصور القادمة من الكاميرا إلى دالة المعالجة بشكل متتابع.

وبذلك يمكن للنظام تحليل صور المخطوطات أمام الكاميرا وإظهار:

- الصورة مع مربعات التحديد.
- الحروف المكتشفة.
- النص المستخرج بشكل لحظي.

وهذا جعل النظام أكثر قابلية للاستخدام من مجرد تحليل ملفات الصور الثابتة.

20. إعدادات المستخدم:

توفر الواجهة إعدادات متقدمة تسمح للمستخدم بالتحكم في بعض خصائص عملية الكشف.

عتبة الثقة (Confidence Threshold): تحدد الحد الأدنى لثقة النموذج حتى يتم قبول الحرف المكتشف. القيمة الافتراضية 0.15 والمدى 0.05 – 0.09.

نسبة التباعد بين الأسطر (Line Spacing Ratio): تستخدم للمساعدة في تحديد ما إذا كانت الحروف تنتمي إلى السطر نفسه. القيمة الافتراضية 0.6، والمدى 0.2 – 1.5.

21. نشر النظام:

بعد تجهيز النموذج وملف التطبيق، تم رفع ملفات التشغيل إلى منصة Hugging Face وإنشاء تطبيق Space /لنشر النظام OCR.

شملت الملفات الأساسية المستخدمة في التشغيل:

- best.pt .
- app.py .
- Alphabet.json .

وبذلك أصبح النظام متاحًا من خلال رابط على الإنترنت بدل أن يكون تشغيله مقتصرًا على بيئة التطوير المحلية.

22. تحويل النظام إلى تطبيق للهاتف:

بعد نشر النظام على Hugging Face والحصول على رابط الواجهة، تم استخدام موقع Web Into App لتحويل رابط الويب إلى تطبيق يمكن تشغيله على الهاتف.

وبهذه الطريقة أصبح المستخدم قادرًا على الوصول إلى نظام OCR من خلال تطبيق على الهاتف، دون الحاجة إلى تشغيل ملفات Python أو بيئة التطوير بشكل مباشر.

23. البنية النهائية للنظام:

يمكن تلخيص البنية النهائية للمشروع بالشكل التالي:



24. النتائج:

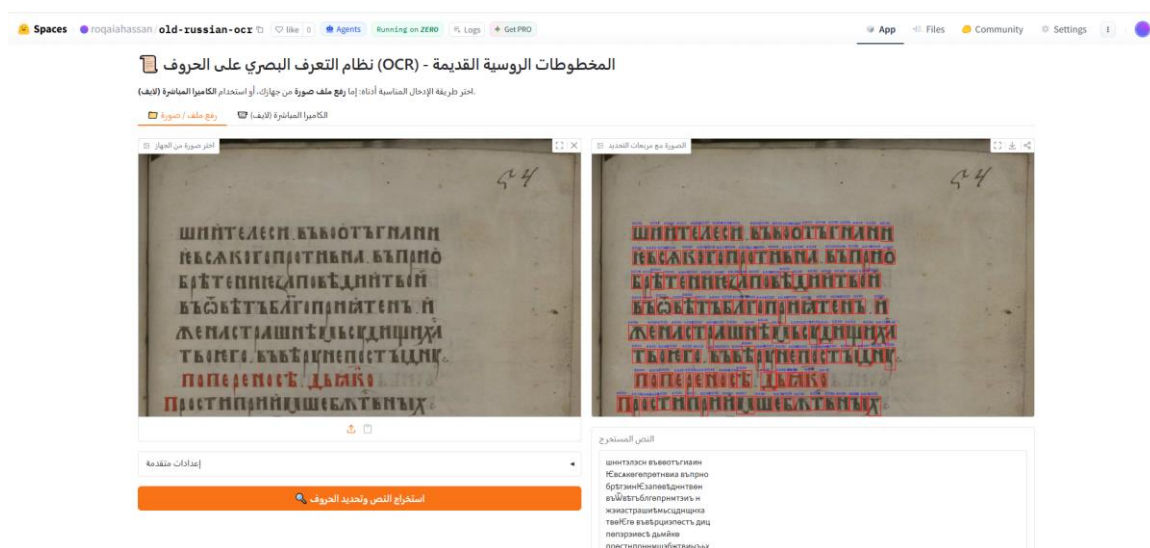
بعد تنفيذ مرحلتي التدريب والضبط الدقيق، حصل النموذج على النتائج التقريبية التالية:

النتيجة التقريبية	المقياس
من 72% إلى 75%	دقة الصناديق المتوقعة (Box Precision)
من 73% إلى 74%	معدل الاسترجاع (Box Recall)
حوالي 67.7%	متوسط الدقة (mAP@50)
من 43% إلى 44%	متوسط الدقة (mAP@50-95)

تشير هذه النتائج إلى أن النموذج تمكن من اكتشاف نسبة جيدة من الأحرف الموجودة في الصور، كما استطاع تحديد مواقعها داخل الوثائق بدرجة مقبولة.

ويعد الوصول إلى 67.7% في mAP@50 نتيجة مشجعة، خصوصًا بالنظر إلى محدودية عدد الصور الحقيقية، وصعوبة طبيعة الوثائق، وتنوع أشكال الحروف، ووجود التشويش والتلف في بعض الصور.

كما أظهرت النتائج أن مرحلة التدريب المسبق بالصور الاصطناعية ساعدت في تهيئة النموذج، بينما ساعدت مرحلة الضبط الدقيق بالصور الحقيقية على تقريب أداء النموذج من طبيعة الوثائق التاريخية الفعلية.



25. التحديات التي عالجها المشروع:

واجه المشروع عددًا من التحديات المرتبطة بطبيعة المخطوطات القديمة، ومن أهمها اختلاف الخطوط وأشكال الحروف، وجود الضوضاء والخدوش والبقع، اختلاف الخلفيات، وتفاوت جودة الصور. ولمعالجة هذه التحديات تم الاعتماد على مجموعة من الحلول، أهمها إنشاء بيانات اصطناعية متنوعة، استخدام أكثر من نوع من الخطوط والخلفيات، إضافة تأثيرات تشبه ظروف المخطوطات الحقيقية، ثم استخدام بيانات حقيقية في مرحلة Fine-tuning كما تم استخدام معالجة مسبقة للصور الحقيقية لتقليل الضوضاء وتحسين التباين.

26. الخلاصة:

تم في هذا المشروع تطوير نظام متكامل للتعرف الضوئي على حروف المخطوطات الروسية القديمة باستخدام تقنيات التعلم العميق ونموذج YOLO. بدأ العمل بإعداد أبجدية مكونة من 38 فئة، ثم إنشاء Dataset اصطناعية تحتوي على حوالي 19,000 صورة، مع تنوع الخطوط والخلفيات والتأثيرات وإنشاء Bounding Boxes للحروف تلقائيًا. بعد ذلك تم تدريب نموذج YOLO على البيانات الاصطناعية، ثم تحسينه باستخدام صور حقيقية من المخطوطات من خلال مرحلة Fine-tuning، مع تطبيق عمليات معالجة للصور لتحسين جودتها. بعد الحصول على النموذج النهائي، تم تطوير واجهة باستخدام Gradio تسمح برفع الصور أو استخدام الكاميرا المباشرة، ثم يقوم النموذج باكتشاف الحروف وتحديد مواقعها وإعادة ترتيبها وتجميعها في أسطر لإنتاج النص المستخرج. وفي المرحلة النهائية تم نشر النظام على Hugging Face، ثم استخدام خدمة Web Into App لتحويل رابط النظام إلى تطبيق يمكن استخدامه على الهاتف. وبذلك انتقل المشروع من مرحلة إعداد البيانات والتدريب إلى نظام OCR قابل للاستخدام من خلال واجهة ويب وتطبيق على الهاتف، مع إمكانية إدخال صور المخطوطات أو استخدام الكاميرا لاستخراج النص.